Activation Planner — Decisions Log

Read this alongside Activation_Planner_Kickoff_Memo.pdf and Activation_Planner_Reference.pdf. This is a living document, updated across brainstorming sessions, tracking the Section 11 open items from the Reference doc through to resolution. Re-upload this file at the start of each new session so decisions aren't lost or re-litigated from scratch.

Legend: 🟢 Jim's idea | 🔵 Claude's suggestion/research — not a decision | 🟣 Decided together, treat as settled | 🟠 Still open, not yet decided

Working agreement: No code is written until planning is complete. Items are worked through one at a time, thoroughly, to a real conclusion — not left hanging. This log is expected to grow (new items added) and shrink (items resolved or dropped) as sessions continue.

Status Overview — Section 11 Open Items

#	Item	Status
1	Relationship to EMMCOM Field Comms Server / IcomRigControl	🟣 **SETTLED**
2	EMCOMM go-kit checklist + NVIS/regional propagation framing	🟣 **SETTLED**
3	VOACAP actual license terms	🟣 **SETTLED**
4	Planning-only, no logging duplication boundary	🟣 **SETTLED**
5	UI framework	🟣 **SETTLED**
6	POTA API (endpoints, auth, rate limits)	🟣 **SETTLED** (scope + approach; exact endpoint/schema research still pending, non-blocking)
7	Mobilinkd TNC4 / Digirig Mobile / SignaLink USB placement	🟣 **SETTLED**, plus new Mission Type concept introduced
8	Antenna modeling staging (Option A vs B)	🟣 **SETTLED**, with one sub-decision deferred pending test data
9	Antenna-category-mapping approach for Option A	🟣 **SETTLED**
10	Begin architecting (solution structure, class library, UI	🟣 **SETTLED**

#	Item	Status
	layer)	
11	Export/share a plan	🟣 **SETTLED**
12	Historical propagation vs. actual conditions trend view	🟣 **SETTLED**
13	Sunrise/sunset and grey-line indicator	🟣 **SETTLED**
14	“Quick mode” for in-field fast replanning	🟣 **SETTLED**
15	Multi-park/route planning	🟢 Accepted in concept — **deferred to v2.0**, not in current build

Item #1 — Relationship to IcomRigControl / FieldCommand IMS 🟣 SETTLED

Correction to prior understanding: The original Reference doc (Section 9.2) described this relationship as being with “the EMMCOM Field Comms Server integration already built into IcomRigControl (Phase 6, complete)” — implying a feature living inside IcomRigControl. This was incorrect. Jim has since shared the FieldCommand IMS Complete User Manual: **FieldCommand IMS is Jim’s own, entirely separate, standalone incident management platform** — an offline-first ICS/NIMS all-hazards system running on its own Raspberry Pi 5 server with its own network (EMCOMM-NET), its own 48-tool web suite, and its own GitHub repository (MIT-licensed source, CC BY-SA 4.0 docs). It has no dependency on, or connection to, IcomRigControl’s codebase.

Relevant background (Jim, for context on why this decision carries real weight): 30+ years with USDA NRCS as a District Conservationist (land use planning); 30-year volunteer Deputy Branch Director with McHenry County Emergency Management Agency; 25-year EMT with a fire department. FieldCommand IMS was built specifically because Jim observed that a sizable portion of emergency managers don’t adequately correlate good communications with effective all-hazards ICS — the platform natively integrates amateur and public-safety communications directly into ICS/IAP development specifically to force that connection to be visible.

Decision: Activation Planner remains **fully separate** from both IcomRigControl and FieldCommand IMS. No shared codebase, no required technical integration between any of the three.

Reasoning — each program has a genuinely distinct intent, not just a distinct feature set: - **IcomRigControl:** in-the-moment rig operation and logging. - **FieldCommand IMS:** incident-level command, coordination, and accountability — scoped to the incident and the organization. - **Activation Planner:** pre-operation (and re-invokable) *individual* planning — what band, what antenna, what gear — scoped to the operator and their personal gear, grounded in real propagation science.

Keeping the three separate preserves this clarity of purpose: FieldCommand IMS stays focused on incident command without absorbing individual antenna-modeling minutiae, and Activation Planner stays focused on being genuinely excellent at personal propagation/gear planning without becoming a second incident management system.

Item #2 — EMCOMM Go-Kit Checklist Content 🟣 SETTLED

(Propagation-framing half was settled earlier as part of Item #7 — mission type driving NVIS/regional CIRCUIT framing. This section covers the remaining piece: actual go-kit content.)

Scope: Personal go-kit only — for the individual operator. Does not track shared/team gear or resources; that remains FieldCommand IMS's domain (consistent with Item #1's separation of concerns).

Category structure (loosely modeled on FieldCommand IMS's Preflight Checklist categories — Power / Communications / Logistics-Supplies / Safety — scaled down from org-level readiness to one operator's personal kit):

Category	Example Items
Power	Extra/spare battery, battery charger, solar panel, DC power cables, fuses, power distribution strip
Communications (beyond primary radio gear)	Backup/spare antenna, headset, spare coax/adapters, hand mic — plus, for EMCOMM specifically, the Digirig Mobile / SignaLink USB / Mobilinkd TNC4 decision from Item #7, since their mission-type-driven priority (optional for POTA/SOTA, potentially load-bearing for EMCOMM) applies directly here rather than being a separate system
Documentation & Traffic Handling	ICS-213/214 reference pad or printed forms, NTS radiogram pad, pen/pencil, notepad, clipboard
Personal Safety/Comfort	First aid kit, water, weather protection, headlamp/flashlight
Identification/Coordination	Copy of license, ID, emergency contact card

Mechanism: Follows the same owned-first, suggest-to-acquire-second rule established in Item #7. For each template item, the Planner checks the operator's inventoried gear — owned items are suggested as "pack this," unowned items become a secondary "consider acquiring" note, kept clearly separate from the primary list.

Item #3 — VOACAP Actual License Terms 🟣 SETTLED

Found directly at source: `its.ntia.gov/software/high-frequency/voacap-propagation-model` (NTIA/ITS official page). Exact disclaimer text:

> "NTIA does not make any warranty of any kind, express, implied or statutory, including, without limitation, the implied warranty of merchantability, fitness for a particular purpose, non-infringement and data accuracy… You can use, copy, modify, and redistribute the NTIA-developed software upon your acceptance of these terms and conditions and upon your express agreement to provide appropriate acknowledgments of NTIA's ownership of and development of the software by keeping this exact text present in any copied or derivative works."

Interpretation: - Not a formally named license (no MIT/GPL/BSD label), but NTIA's own permissive terms — explicitly permits use, copy, modify, and redistribute. - Sole condition: if the software (or a derivative) is copied/redistributed, the disclaimer text must be kept present. This condition only triggers if VOACAP's own code is being distributed. - **Confirms the Section 4.3 "mere aggregation" analysis was correct**, now backed by primary source rather than inference: since Activation Planner never copies, modifies, or redistributes VOACAP's code — it only shells out via `Process.Start` to a copy the end user installs themselves directly from NTIA — none of NTIA's redistribution conditions apply to Activation Planner or affect its own AGPLv3/GPLv3 licensing choice.

Item #4 — Planning-Only Boundary 🟣 SETTLED

- The Planner does **planning**, not logging. QSO logging stays in IcomRigControl.
- **The boundary is functional, not temporal.** "Planning mode" can be invoked again mid-activation (e.g., replanning when a band goes dead), not just pre-trip. This is a deliberate refinement beyond the original suggestion.
- **Replanning is stateless.** No session/activation history is tracked. Each replan query uses current time + current solar data and produces a fresh ranked assessment of all bands. The operator applies their own judgment (they know what they just tried).
- **Manual backup:** a per-query "exclude this band" toggle, for cases where the model's ranking doesn't match what the operator is observing on the air. Not persisted — a one-off filter per run.
- **Location is refresh-on-demand.** A "refresh my location" action re-fetches GPS whenever the tool is opened or the operator requests it. Not a one-time locked entry, not continuous background tracking.

Item #5 — UI Framework 🟣 SETTLED

- **Framework:** Avalonia (cross-platform, familiar from IcomRigControl).
- **Visual identity:** Its own distinct look — not a reskin of IcomRigControl. Each program gets its own unique presentation.
- **Visual bar:** Professional, rich, graphical — genuinely pleasant on repeated use, not merely functional.

- **Maps:** Interactive embedded map (pan/zoom). Accepted tradeoff: this implies an internet-connectivity dependency, which is fine because Activation Planner is not primarily an EMCOMM-first, must-work-offline tool — EMCOMM is a supported use case, not the core design constraint.
- **Propagation/band data:** Combination view — list **and** graphical (e.g., ranked list alongside a chart/heatmap-style visual), not one or the other.
- **Checklists:** List-based core interaction, enhanced with grouped categories, progress indicators, and category icons — not a flat checkbox list.

Item #6 — POTA API Integration 🟣 SETTLED (corrected this session)

Research findings (original session): - Official API docs (`docs.pota.app/api/`) are an unfinished stub — no formal published spec, endpoints, or rate limits. - Reading data (spots, park activation history) is an open, unauthenticated GET API already used by multiple established community tools (HAMRS, Log4OM, hunterlog). Low risk. - No published rate limit; community norm is self-throttling out of courtesy, not a documented requirement. - Several third-party devs have noted uncertainty about whether POTA officially sanctions third-party automated use at all.

Correction (this session) — self-spotting does NOT require authentication. Verified directly against `hunterlog`'s actual production source code (`src/pota/pota.py`, an established open-source POTA hunting tool). Real confirmed endpoints:

```
GET  https://api.pota.app/spot/activator              — live spot feed
GET  https://api.pota.app/spot/comments/{act}/{park}  — comments on an
activation
GET  https://api.pota.app/stats/user/{call}            — activator
stats
GET  https://api.pota.app/park/{park}                  — park info
GET  https://api.pota.app/location/parks/{loc}         — parks by
location
GET  https://api.pota.app/programs/locations/          —
program/location reference data
POST https://api.pota.app/spot/                        — post/re-spot
an activation
```

The `POST /spot/` call sends a plain JSON body (`activator`, `spotter`, `frequency`, `reference`, `mode`, `comments`) with ordinary headers — **no `Authorization` header, no bearer token, no cookies.** Self-spotting is simply this same call with `spotter` set equal to `activator`.

Why the earlier Cognito finding was a false lead: the Cognito SRP authentication we found originally belongs to a *different* POTA operation — **uploading a submitted activation log for official award credit**, which legitimately needs to be tied to an authenticated account. Spotting is informational only (not tied to award credit or the permanent record), so POTA doesn't require identity verification to accept one — consistent with POTA's honor-system design overall.

Decisions (revised): - **Reading data (spots, park info): in scope**, low risk, build with confidence — endpoint list now confirmed above. - **Self-spotting: in scope, and significantly simpler than originally designed.** No login flow, no credential handling, no Cognito complexity. A plain unauthenticated `POST` with the fields above. The earlier login → post → discard design and its credential-storage caution are **no longer needed** for this feature. - **Pre-ship checkpoint still stands:** before shipping, reach out to POTA directly to confirm they're comfortable with third-party automated self-spotting — same due-diligence pattern used for the VOACAP automation-policy check. (Several devs have flagged uncertainty about official sanctioning even though the endpoint itself is technically open.) - **Still open (non-blocking, implementation-level):** exact request/response JSON schemas for the remaining endpoints (stats, park info, comments) — straightforward to confirm once building against them directly.

Item #7 — Digital Interface Hardware Placement 🟣 SETTLED (+ new Mission Type concept)

Digirig Mobile vs. SignaLink USB — confirmed real technical differences, not redundancy:

	SignaLink USB	Digirig Mobile
PTT method	VOX-based only	Hard PTT via RTS serial or CAT command
CAT control	Not CAT capable	Full CAT interface (PTT, band, tuning)
Isolation	Built-in (audio transformers)	None built-in; addable via ~$20 USB isolator
Portability	Larger, more physical controls	Smaller, built for field ops
PTT latency	VOX has inherent lag	RTS/CAT avoids VOX latency

- Both are modeled as **distinct gear inventory items**, not one generic "digital interface" entry.
- "Which interface to pack" is a genuine planning decision point tied to: which radio is being brought, whether that radio supports CAT, and whether portability or isolation matters more for that trip.
- This is a planning decision (what to pack), not an inventory reflection (what's in the bag) — consistent with Item #4.

Mobilinkd TNC4: - Confirmed via manufacturer + user sources: **no built-in GPS.** It is a "dumb" TNC — it relies on a paired phone/tablet's GPS via Bluetooth + an APRS app (e.g., APRSdroid). (Note: one low-quality resale listing claimed built-in GPS; rejected in favor of manufacturer/firsthand sources.) - Priority varies by mission type — optional/nice-to-have for POTA/SOTA (position beaconing), potentially load-bearing for EMCOMM (team position tracking, packet messaging).

New concept introduced this session — Mission Type: - The Planner asks the operator to select a **mission type** up front (POTA, SOTA, Field Day, EMCOMM, general/casual, etc.). - This is effectively a way of selecting which **checklist Template** to start from (ties into the Template-vs-Instance model in Reference doc Section 7). - **Gear suggestions draw primarily from the operator's own owned inventory** (entered during a setup session), matched against what the mission type typically needs. - **Secondary layer:** if the mission type typically calls for something not owned, the Planner may note it as a suggested-to-acquire item — kept clearly separate from the primary "pack this" list, never mixed in as if already available. - Operator can freely add/remove from either layer. - **Mission type also drives propagation framing, not propagation results.** Selecting EMCOMM defaults the CIRCUIT setup to regional/NVIS-style framing (near-in receive point representing a served agency/net) rather than DX-style framing. This changes what *question* is asked of VOACAP, never the physics-based answer — no band is ever assumed to work; it's still computed from real current solar/time data. Operator can adjust the framing manually regardless of mission type.

Item #8 — Antenna Modeling Staging 🟣 SETTLED (one sub-decision deferred)

- **Option A (community library)** is the default/first attempt.
- **Option B (NEC2++ custom modeling)** triggers automatically per the rules below — this is a decision rule, not a fixed roadmap phase.

Trigger rules (research-based, see below for confidence level):

Verticals/ground-planes — research-backed: - Convert actual entered height to wavelengths (λ) at the band being evaluated. - If actual height falls within **0.25λ–1.25λ** (documented distortion zone — energy radiates at poor intermediate angles, 27°–45°, regardless of radial count), trigger Option B regardless of library file similarity. - Outside that zone (very low/near-ground-mounted, or genuinely >1.25λ), a library match is more likely valid.

Dipoles/horizontal wire — reasoned threshold, provisional: - Convert both the library file's assumed height and the operator's actual height to wavelengths at the band being evaluated. - If they differ by more than **~0.05λ (≈5%)** at that band, trigger Option B. - Deliberately conservative — errs toward custom modeling when in doubt, since dipole pattern shape changes continuously with height-in-wavelengths rather than at one sharp cutoff. - **Not yet empirically validated.** Planned as a TDD-style task once the VOACAP shell-out exists: build a comparison harness, run the same dipole at varying heights, compare VOACAP's actual reliability/output numbers, and tune the 0.05λ figure based on where predictions actually diverge meaningfully.

Key supporting research: - Height sensitivity is relative to wavelength, not absolute feet — the same physical height difference (e.g., 5 ft) is a large wavelength-fraction on a high band (10m) and nearly negligible on a low band (80m). - Worked example: a fixed 20 ft physical dipole height translates to ~0.08λ on 80m but ~0.61λ on 10m — spanning across the point where dipole pattern behavior meaningfully shifts (~0.25λ). - Ground conductivity affects vertical takeoff angle independently of height (better ground → lower takeoff angle).

Deferred pending test data: whether Option A/B selection should happen **per band/antenna combination** (more accurate — the same physical antenna can genuinely need Option A on one band and Option B on another, since electrical height is a per-band question) versus **once per antenna for the whole session** (simpler, less precise). Revisit once the dipole empirical test shows how much the mismatch actually moves VOACAP's output.

Item #9 — Antenna-Category-Mapping Approach 🟣 SETTLED

Gear entry data model (per antenna, captured at setup): - Category (vertical/whip, EFHW, dipole, magnetic loop, etc.) - Length - Height above ground - **Feed point type** — where/how the feedline connects (center-fed, end-fed/EFHW, end-fed random wire, off-center-fed, base-fed, etc.). Matters both for accurate modeling (NEC needs to know exactly where the excitation source sits on the wire) and for correctly distinguishing antennas that share a category name but behave differently. - **For verticals specifically:** radial count and radial length (operator-entered — not something reliably known/measurable by the operator otherwise, e.g. ground conductivity, so this stays a simple physical count/length the operator can just look at and enter).

Key correction from this session — category name alone is not sufficient for matching. An "EFHW" is not one antenna — a 40m EFHW (~66 ft) and an 80m EFHW (~132 ft) share a category label but are electrically very different. This means **length has the same problem height had in Item #8**, and gets the same treatment: - Convert actual entered length to wavelengths at the band being evaluated. - Compare against what the library file assumes. - If the mismatch exceeds a similar tolerance to the height rule, trigger Option B (NEC2++) — independent of, and in addition to, the height-based trigger already established in Item #8.

Ground conductivity — auto-looked-up, not operator-entered. Research finding: the FCC maintains real, downloadable ground conductivity data covering the continental US (plus Hawaii and the rest of North America), available as text/KML files keyed by location — not something the operator needs to know or guess. Approach: - Look up conductivity automatically from the antenna's actual location (using the GPS/location data already gathered elsewhere in the app). - Map the looked-up conductivity to the nearest standard VOACAP ground preset (good/average/poor), which pairs conductivity with a matching permittivity value — since the FCC dataset only captures conductivity, not permittivity, and VOACAP's ground model wants both. - Honest caveat carried over from research: FCC data is built for the AM broadcast band and technically needs some correction for HF/ground-wave use — treated as a strong practical default, not a lab-grade measurement. - Consistent with the project's established principle (from the VOACAP automation check onward): trust real data over operator guesswork wherever a real data source exists.

Fallback when nothing fits well: confirmed — if a gear entry doesn't map well to any community category (unusual/homebrew design), or if length/height mismatches exceed tolerance, the system routes to **Option B (NEC2++)**, consistent with the Item #8 decision rule rather than a separate "closest guess with a warning" fallback.

Item #10 — Architecture (IN PROGRESS)

Source pattern: Adapted directly from IcomRigControl's proven layered architecture (never mix concerns across layers), per Jim's own project spec:

- **Layer 1 — `ProcessEngine`** (parallel to IcomRigControl's `CivEngine`): Raw external-process I/O — shelling out to VOACAP and NEC2++ via `Process.Start`, writing input decks/geometry files, reading raw text output. No domain knowledge, no UI.
- **Layer 2 — `PropagationModel`** (parallel to `RigModel`): Clean C# domain classes (`BandPrediction`, `AntennaProfile`, `CircuitQuery`, `GearItem`) exposing ProcessEngine's raw output as real objects/events. Consumes ProcessEngine only.
- **Layer 3 — `Services`**: Multiple peer services, each independent, all consuming Layer 2 where relevant:
  - `ChecklistService` — Template/Instance checklist logic (Reference doc Section 7)
  - `MissionTypeService` — Mission Type selection and gear/propagation-framing suggestions (Item #7)
  - `GearInventoryService` — owned gear, antenna category-mapping, Option A/B trigger logic (Items #8–#9)
  - `PotaService` — **decided this session**, internally split into: (a) simple read-only spot/park data client, and (b) the auth-sensitive login→post→discard self-spotting flow (Item #6). Kept as its own sub-module rather than folded into general logic, since it carries a distinct risk profile (real network dependency, Cognito-based auth complexity) — mirrors how IcomRigControl's Services layer already holds multiple distinct peer services (Logger, EMMCOM bridge, APRS beacon) rather than merging them.
- **Layer 4 — `UI`**: Avalonia views/viewmodels. Consumes Services and PropagationModel only.

Coding standards — adopted directly from IcomRigControl, no divergence: - .NET 10 / C# 12, nullable reference types enabled - async/await for all I/O (process shell-outs and POTA HTTP calls), CancellationToken threaded through - Records for immutable data (`BandPrediction`, `AntennaProfile`, `CircuitQuery`, `GearItem`, `MissionTemplate`) - No magic numbers — VOACAP card format constants and NEC2++ geometry constants centralized - **Critical carryover:** all process shell-out access goes through an `IProcessTransport`-style interface (mirroring `ICivTransport`) — makes the VOACAP/NEC2++ dipole-height empirical testing from Item #8 mockable/testable without requiring real VOACAP installed on the test-running machine - Unit tests required for: VOACAP input deck formatting, output parsing, and the Option A/B trigger logic (height/length-to-wavelength comparisons)

Solution structure — CONFIRMED, matching IcomRigControl's actual repo layout (verified directly): separate `.csproj` per layer, each in its own folder — `ActivationPlanner.ProcessEngine`, `ActivationPlanner.PropagationModel`, `ActivationPlanner.Services`, `ActivationPlanner.UI`, plus test projects per layer. Compiler-enforced boundaries via `ProjectReference`, not just convention — matches IcomRigControl's own pattern.

“Do Not” rules, adapted directly from IcomRigControl’s spec: - Do not implement features out of phase order without explicit instruction - Do not add NuGet packages without listing them here first - Do not put VOACAP/NEC2++ shell-out logic in ViewModels or Services - Do not put UI code in ProcessEngine or PropagationModel - Do not use `Thread.Sleep` — use `Task.Delay` with `CancellationToken` - Do not swallow exceptions silently

Build phase order — SETTLED: 1. **Gear inventory** — built first, since antenna/band matching needs real gear data to work against 2. VOACAP shell-out (ProcessEngine + PropagationModel) 3. Antenna category-mapping + Option A/B trigger logic (Items #8–#9) 4. Avalonia UI — core planning screen (band/antenna recommendations, list+chart view) 5. Checklist/Template engine + Mission Type selection 6. GPS/location integration 7. POTA integration (read-only spots/park data first; self-spotting later, pending the POTA confirmation checkpoint from Item #6) 8. NEC2++ shell-out (Option B custom antenna modeling)

Gear inventory setup flow — SETTLED: A guided, **required** first-use wizard walks the operator through entering their owned gear before other features are used — not an on-demand/optional prompt. Data persists and remains **fully editable afterward** — add, delete, or amend at any time, not just during initial setup.

Gear Inventory Setup Wizard — UI pattern SETTLED: - Classic step-by-step wizard: one category of input per step (e.g., Radios, Antennas, Power/Digital interfaces, EMCOMM-specific gear), **Back/Next** navigation, visible progress indicator (“Step 3 of 6”), and a **Finish** summary step before committing data. - Steps can be **skipped** if the operator owns nothing in that category — doesn’t block progress. - **One-time guided entry point only** — since inventory data stays fully editable afterward (per the earlier setup-flow decision), the wizard isn’t re-run later; post-setup editing happens through a regular, non-wizard inventory management screen. - **Antenna step specifically uses a sub-list-and-detail pattern**, not a flat multi-field form and not a separate wizard step per antenna: the operator adds one antenna at a time via a focused mini-form (category, length, height, feed point type, plus radial count/length for verticals), saves it, and it appears as a row in a running list. Add/edit/remove any row, then proceed when done. Chosen specifically to keep each individual entry simple while still handling a variable number of owned antennas — same list-with-detail-view shape as FieldCommand IMS’s T-card personnel tab pattern.

MVVM Pattern — SETTLED: CommunityToolkit.Mvvm, matching IcomRigControl exactly (confirmed directly from IcomRigControl source files: `ViewModelBase : ObservableObject`, `[ObservableProperty]` for bindable fields, `[RelayCommand]` for commands) — not ReactiveUI.

Test project layout — SETTLED: One test project per production layer (`ProcessEngine.Tests`, `PropagationModel.Tests`, `Services.Tests`), mirroring the production project structure rather than one consolidated test project — preserves the same compiler-enforced layer boundaries in tests that the production code gets, rather than letting tests casually reach across layers.

Required tests (targeted list, not blanket coverage — matching IcomRigControl’s own approach of naming specific error-prone pieces): - VOACAP input deck writer (fixed-column formatting — flagged as bug-prone in the Reference doc) - VOACAP output parser - Option A/B antenna trigger logic (height/length-to-wavelength math, Items #8–#9) - Dipole empirical

comparison harness (Item #8's planned validation work) - POTA auth flow (login → post → discard)

Item #10 is now fully settled — solution structure, layer architecture, coding standards, build phase order, gear inventory setup flow and wizard UI pattern, MVVM framework, and test project layout are all decided.

Item #11 — Export/Share a Plan 🟣 SETTLED

Concept: Export the current plan (bands, antenna recommendation, checklist) as PDF — for printing, sharing, or hand-transcribing into an ICS-213 for EMCOMM use. Reads from the same PropagationModel/Services data already produced for the UI; no new architecture needed.

Format: PDF only (not plain text).

Content: Operator-selectable — choose which sections to include (band recommendations, antenna pick, checklist, any combination) rather than a fixed always-everything export.

Trigger: A dedicated **Export** button on the main planning screen.

Item #12 — Historical Propagation vs. Actual Conditions Trend View 🟣 SETTLED

Concept: A quick trend view — "what did VOACAP predict recently vs. now" — to support the replanning moment (e.g., a band going dead). Purely ephemeral/session-local, no persistence, so it does **not** conflict with the stateless-replanning decision from Item #4 (no activation history is stored; this is just a rolling short-term comparison of recent predictions within the current session).

Window: A rolling few-hour trend — long enough to see meaningful change (e.g., "40m was good an hour ago, now it's fading"), short enough to stay session-local and simple.

Sampling method: Automatic background sampling on a ~15–30 minute interval, running for as long as the app is open — not dependent on the operator manually replanning or checking in. Chosen over capture-on-replan-only because a trend built from evenly-spaced automatic samples gives an honest, continuous picture; a trend built only from whenever the operator happened to replan would be thin and easy to misread as more meaningful than it is.

Item #13 — Sunrise/Sunset & Grey-Line Indicator 🟣 SETTLED

Concept: Compute sunrise/sunset from lat/lon + date and surface a grey-line indicator alongside the VOACAP chart — grey-line propagation is a well-known real DX phenomenon, particularly relevant for 80m/40m evening/morning planning.

Design — informational overlay plus correlation highlight, not a separate ranking input: - The grey-line window is shown as a visual annotation on the propagation chart —

purely descriptive. - Rather than adding a separate score boost for bands during grey-line, the UI highlights when a band **VOACAP already ranks well** happens to **coincide** with the grey-line window — surfacing a real correlation in the existing prediction rather than adding a second opinion on top of it. - **Reasoning:** VOACAP's own physics model already incorporates day/night solar zenith angle and D-layer absorption when computing reliability at a given time — grey-line propagation is, in effect, already partially reflected in its output. A separate artificial "grey-line boost" risks double-counting an effect the real prediction may have already accounted for, which conflicts with the "never let the tool assume a band works — always compute from real data" principle established in Item #7.

Item #14 — "Quick Mode" for In-Field Fast Replanning 🟣 SETTLED

Concept: A fast, minimal-friction path for "I'm already at the park, give me one fast answer" — distinct from the full guided planning flow used for initial trip prep.

Design — fewer taps, not less content: - Quick Mode is a **fast-access entry point** to the same replanning capability already established in Item #4 — not a separate, reduced feature set. - Triggered via a single dedicated action; immediately runs a replan using the current mission type and current/last-known location, skipping intermediate setup screens, landing directly on the band/antenna recommendation view. - **The recommendation content itself is not truncated or simplified** — same full band ranking, same real data. Only the *path to reach it* is shortened, since a "quick" view thin enough to be useless would defeat the purpose of a decision-support tool.

Item #15 — Multi-Park/Route Planning 🟢 Accepted in concept, deferred to v2.0

Concept: Support for operators doing a park-to-park run in one day — planning band/antenna per stop rather than one session at a time.

Status: Deferred to a future v2.0 release — not part of the current (v1.0) build order.

Reasoning for deferral (clarified this session): Items #11–#14 all extend the *existing* single-session planning model without changing its core data shape — export reads existing data, the trend view is a rolling window on existing predictions, grey-line is an overlay, Quick Mode is a faster path to the same replan. Multi-park routing is structurally different: it implies planning across multiple locations and multiple time windows in sequence (e.g., "leave Park A at 2pm, arrive Park B by 4pm — what band/antenna at each, accounting for travel time and how conditions will have shifted by arrival"). That's a genuinely different planning unit — a *route* of sessions, not one session — likely requiring new concepts (inter-stop scheduling/timing, a per-stop plan UI) rather than a bigger version of what's already designed.

Decision: Build and ship v1.0 (single-session planning, Items #1–#14) first, prove it works, then design multi-park/route planning properly as a v2.0 feature once there's a solid, tested

foundation to extend — rather than guessing at a second planning paradigm's design before the first one exists.

Open / Not Yet Discussed

(none remaining — all Section 11 items, #1–#10, are settled)

Document generated during planning session. Update and re-save after each future session before ending.